Restaurant Order Management System

Final Project Report
Database-Backed Application

Course: Database Systems

Track: Database-Backed Application

Database Platform: Oracle

Student Name: Nakulesh Jayakrishnan

Monroe ID: 0295600

Github Repo: https://github.com/nakuleshj/database-systems-project

Summary

This project presents a Restaurant Order Management System designed as a normalized Oracle database application. The database supports customer records, employee roles, menu organization, dining tables, orders, order line items, and payments. The design demonstrates core relational database concepts including entity identification, primary and foreign keys, referential integrity, check constraints, and separation of transactional data from lookup and master data.

The system was intentionally scoped to be realistic but manageable for an academic project. It models a restaurant's daily workflow from order entry through payment while keeping the design normalized to Third Normal Form (3NF). Deliverables for the project include an ER diagram, Oracle SQL scripts for schema creation and sample data, a reset script, and this written report.

Introduction

Restaurants handle several connected business activities at the same time: they manage customers, staff, menu items, dining tables, active orders, and final payments. Without a structured database, this information can become inconsistent, duplicated, or difficult to report on. The purpose of this project is to design a relational database that organizes those operations clearly and enforces data quality through keys and constraints.

The selected business case is a restaurant order management platform because it offers a strong mix of master data and transactional data. It also allows the database design to demonstrate one-to-many relationships, a bridge table to resolve a many-to-many relationship, and practical validation rules such as positive prices, valid employee roles, and permitted order statuses. The final design is both academically appropriate and close to a real operational use case.

Data Modeling

The database contains eight entities: Customers, Employees, Categories, Menu_Items, Dining_Tables, Orders, Order_Items, and Payments. Together, these tables capture the core operational process of the restaurant. Customers and employees represent the people interacting with the business. Categories and menu items represent the products sold. Dining tables represent the physical seating layout. Orders and order items represent the transaction detail, and payments record settlement for completed orders.

Entity	Primary Key	Purpose	Key Relationship
Customers	customer_id	Stores customer contact and loyalty details	Referenced by Orders
Employees	employee_id	Stores staff records and roles	Referenced by Orders
Categories	category_id	Groups menu items by type	Referenced by Menu_Items
Menu_Items	menu_item_id	Stores products sold by the restaurant	Referenced by Order_Items
Dining_Tables	table_id	Stores seating capacity and location	Referenced by Orders
Orders	order_id	Stores order headers and status	References Customers, Employees, Dining_Tables
Order_Items	order_item_id	Stores individual order	References Orders and

		lines	Menu_Items
Payments	payment_id	Stores payment details for orders	References Orders

The most important transactional relationship is between Orders and Menu_Items. A single order can contain multiple menu items, and the same menu item can appear on many different orders. This many-to-many relationship is resolved through the Order_Items table. Each order item stores the specific menu item purchased, the quantity, the price charged at the time of sale, and any special request. This structure prevents repeating groups and allows detailed sales analysis.

ER Diagram

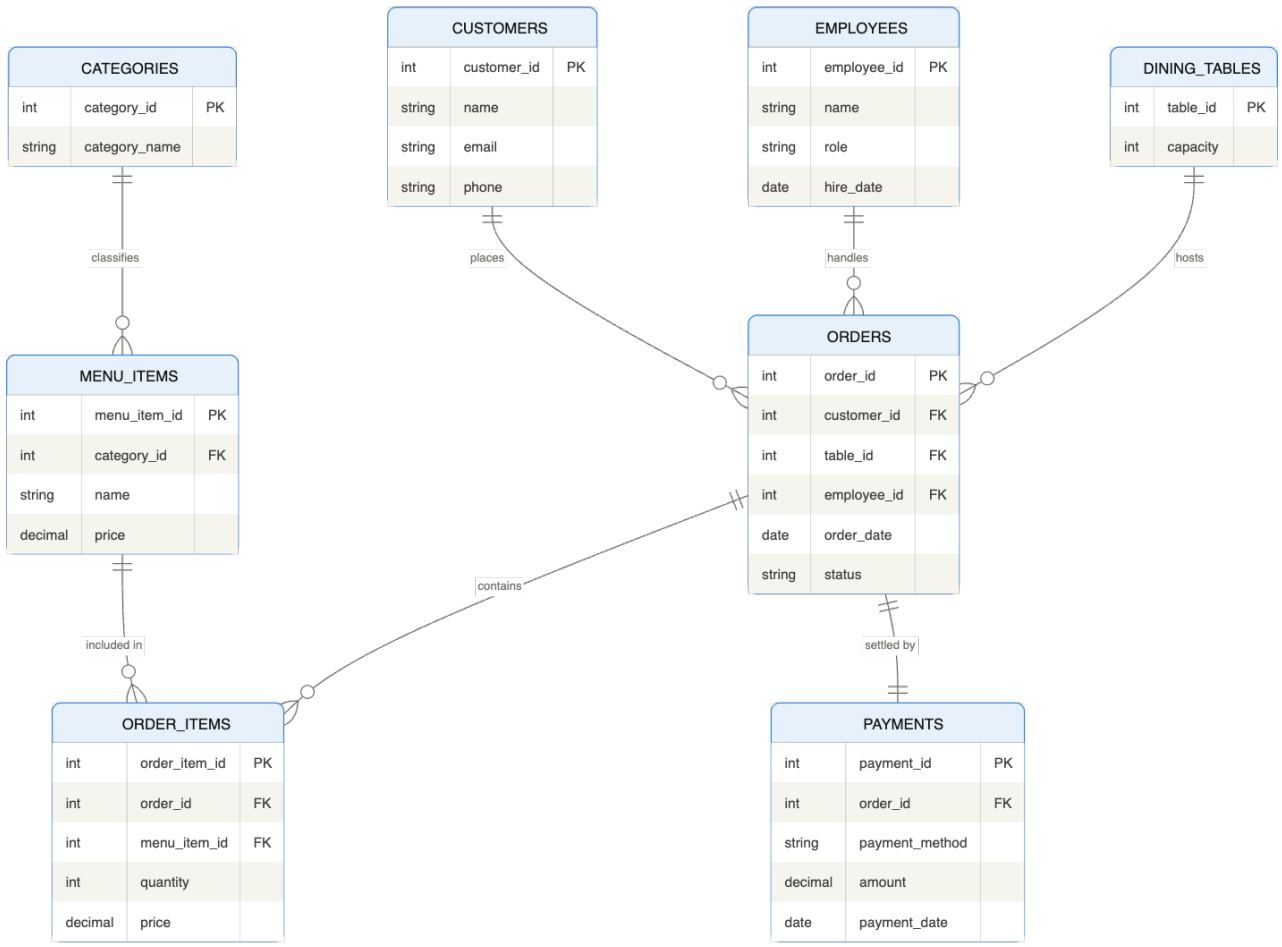

Figure 1. ER diagram showing the core entities, keys, and cardinalities in the Restaurant Order Management System.

Explanation: The ER diagram shows how master data tables connect to the order transaction flow. It also highlights how Order_Items resolves the many-to-many relationship between Orders and Menu_Items.

Design Decisions

Several design choices were made to keep the database clear, scalable, and normalized. First, customer, employee, category, menu item, table, order, and payment data were separated into different tables so that each table serves one clear business purpose. This reduces redundancy and makes updates more reliable. For example, menu category descriptions are stored once in Categories instead of being repeated for every menu item.

Second, the design stores both Orders and Order_Items because order-level data and line-level data represent different levels of detail. Order header data includes the customer, employee, table, date, type, and status. Line item data includes the exact products purchased and their quantities. This separation improves reporting and aligns with standard transaction modeling.

Third, Payments was separated from Orders to allow financial settlement information to be managed independently from order entry. This decision improves extensibility because the design could later be expanded to support partial payments, refunds, or multiple payment events if needed.

Fourth, the schema includes check constraints to improve data quality. Examples include limiting employee roles to approved values, ensuring prices and quantities are positive, restricting order types and statuses to valid business values, and preventing duplicate dining table numbers. These rules make the design more robust and reduce the risk of invalid data.

Finally, indexing was added on commonly queried columns such as order date, order status, foreign key columns, and menu category. These indexes support better query performance for reporting and daily operations.

Normalization to 3NF

The database is normalized to Third Normal Form. Each table has a clearly defined primary key. Repeating groups are removed by storing order lines in a separate Order_Items table. Non-key attributes depend on the key and not on other non-key attributes. For example, category details depend only on category_id, and payment details depend only on payment_id and the associated order. There are no obvious transitive dependencies in the final design.

A practical example is Menu_Items. The item name, description, price, and active status all describe one menu item, while the category details are kept in a separate Categories table. This prevents repeated category names and descriptions across multiple rows. Similarly, Orders stores the transaction header, while Order_Items stores the detailed items included in that transaction.

Screenshots of Database Design

Orders Table Design

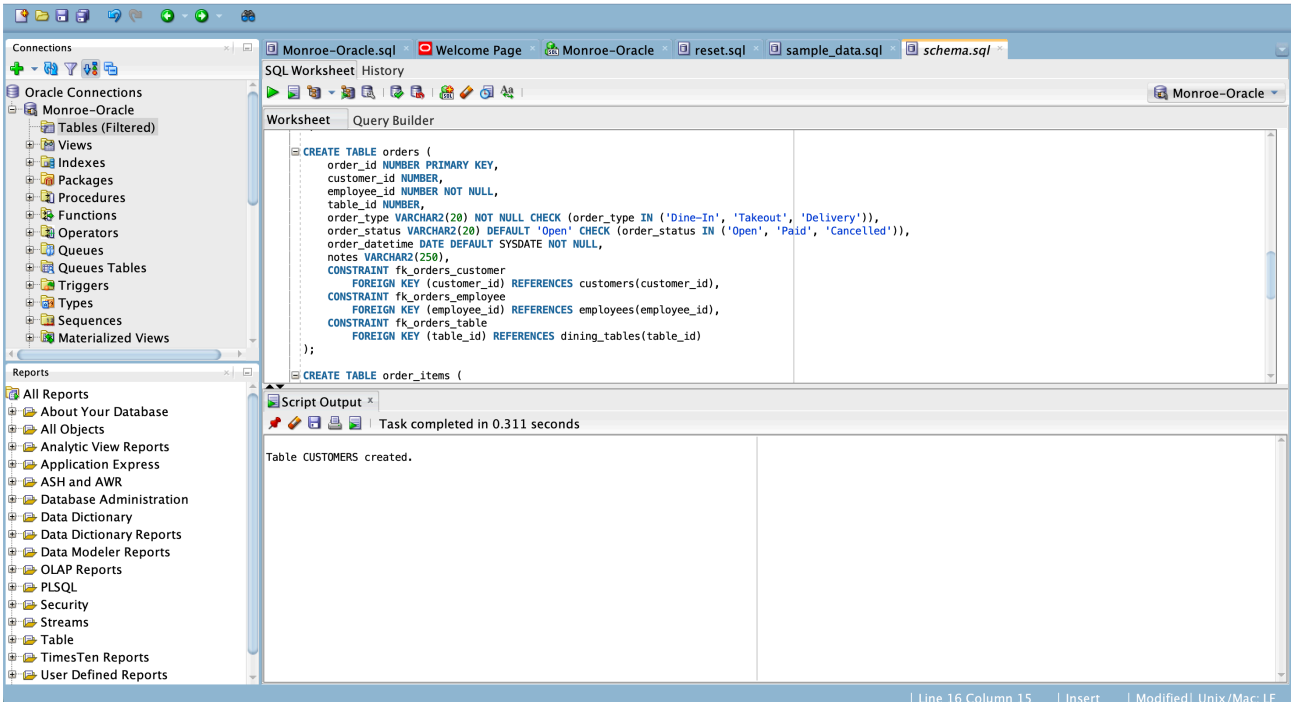

Figure 2. DDL excerpt for the Orders table.

Explanation: This table captures the order header and connects the transaction to the customer, employee, and dining table while enforcing valid order types and statuses.

Order_Items Table Design

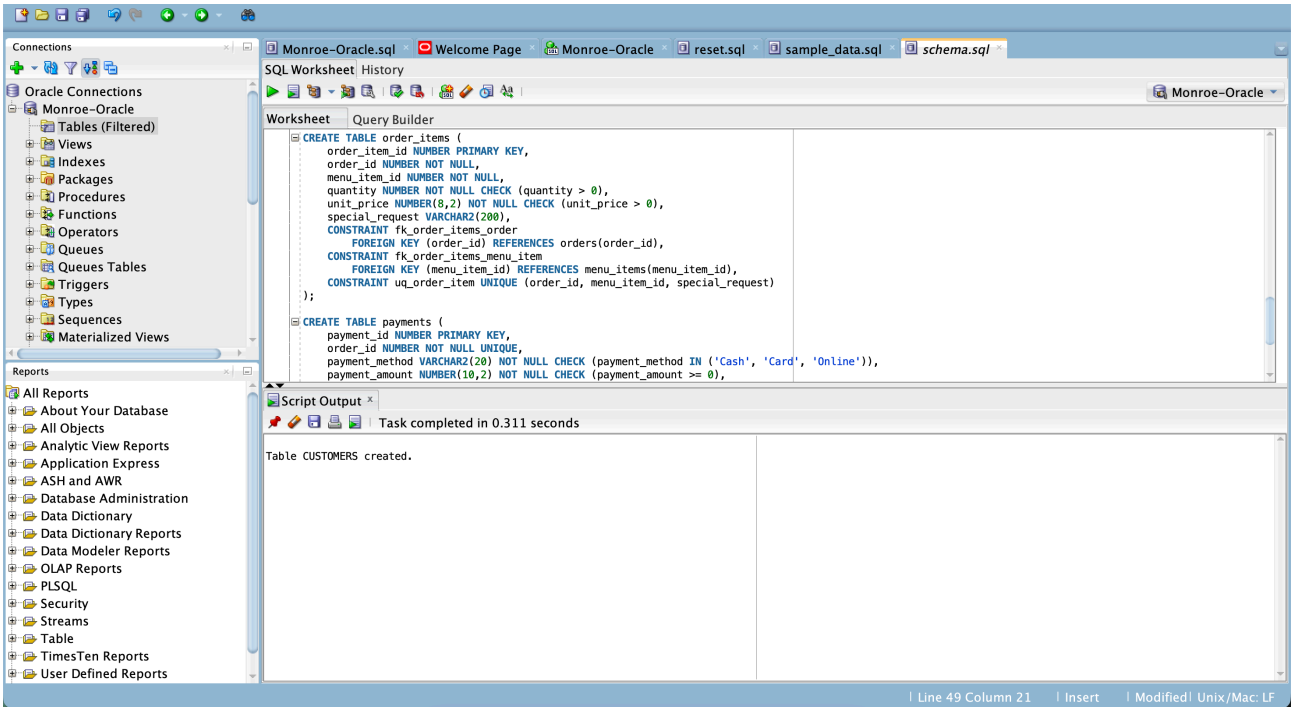

Figure 3. DDL excerpt for the Order_Items table.

Explanation: This table stores each line item within an order and resolves the many-to-many relationship between Orders and Menu_Items.

Implementation Files

The project includes three required SQL files. The schema.sql file contains Oracle DDL statements for table creation, key definitions, constraints, and indexes. The sample_data.sql file inserts representative records across all tables so the database can be tested immediately. The reset.sql file drops the tables in dependency order using Oracle PL/SQL blocks so that the environment can be reset safely before rerunning the schema.

A public GitHub repository should be created for submission. To satisfy the assignment requirement, the repository should show incremental progress with at least ten commits. A clean structure would include a sql folder for scripts, an assets folder for the ER diagram, the final report, and a README that summarizes the business case and project contents.

Conclusion

This Restaurant Order Management System demonstrates a complete database-backed application with a strong balance of realism and clarity. The final design supports operational workflows that are common in the restaurant industry while remaining normalized, maintainable, and easy to explain. The project shows proper use of primary keys, foreign keys, constraints, bridge-table design, and structured sample data.

Overall, the project meets the assignment goals by combining sound data modeling with Oracle implementation. It can also be extended in the future to support reservations, inventory, supplier purchasing, or multiple payment events. In its current form, it provides a strong academic example of database design and data management principles.